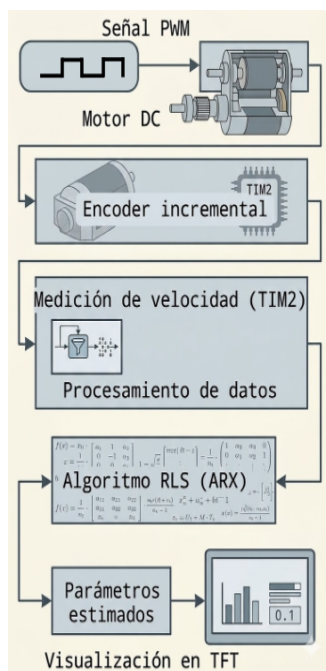


Módulo 6 - Comunicación serial con USART3

Proyecto de identificación RLS/ARX en STM32F405RGT6

Archivos documentados: uart_driver.h, uart_driver.c, uart_if.h y uart_if.c.



Función del módulo. Habilita una interfaz serial asíncrona para transmitir y recibir bytes por USART3. En este proyecto se inicializa a 115200 baudios.

Periférico y pines. Usa USART3 en PB10 = TX y PB11 = RX, ambos en función alterna AF7.

Relación con el sistema. Aunque en main.c solo se invoca la inicialización, este bloque deja lista la infraestructura para depuración, monitoreo de variables del RLS, captura de comandos y registro de datos experimentales.

Patrón de diseño. El archivo uart_if empaqueta las operaciones de transmisión y recepción dentro de una estructura constante, de modo que la aplicación no depende directamente del nombre concreto del driver.

1. Papel del módulo dentro del firmware

Este módulo resuelve la parte más básica y más útil de una comunicación serial embebida: enviar y recibir bytes con un puerto UART real del microcontrolador. En proyectos de identificación de sistemas como este, una interfaz serial suele usarse para observar variables internas, ajustar parámetros durante pruebas o incluso descargar datos para procesarlos en una computadora.

En tu implementación actual, USART3 todavía no participa activamente en el lazo principal del RLS. Sin embargo, ya está inicializado en el arranque, lo cual deja preparado el camino para futuras ampliaciones como imprimir la velocidad medida, enviar los parámetros estimados del modelo ARX, recibir consignas desde una terminal o sincronizar el firmware con herramientas de laboratorio.

2. Relación entre archivos

Archivo	Responsabilidad
uart_driver.h	Declara la API pública del driver: inicialización, envío de byte, envío de cadena, recepción bloqueante, recepción no bloqueante y espera del fin físico de transmisión.
uart_driver.c	Configura relojes, GPIOB y registros de USART3. También implementa las rutinas de envío y recepción mediante polling.
uart_if.h	Define una estructura de interfaz con punteros a función para desacoplar la aplicación de la implementación concreta del driver.
uart_if.c	Construye la instancia constante <code>uart_STM32</code> enlazando cada operación de la interfaz con la función real del driver.

Fragmento 1. API publicada por uart_driver.h

```
void usart3_pb10_pb11_init_115200(void);
void usart3_write_byte(uint8_t b);
void usart3_write_str(const char *s);
int usart3_read_byte_nonblocking(uint8_t *out);
uint8_t usart3_read_byte_blocking(void);
void usart3_flush_tc(void);
```

3. Fundamento serial y mapa de hardware

UART significa Universal Asynchronous Receiver/Transmitter. Se trata de un enlace serial asíncrono: no existe una línea de reloj compartida entre emisor y receptor, por lo que ambos extremos deben coincidir en baudios, longitud de palabra, paridad y bits de parada. En este módulo se usa la configuración clásica 8N1: 8 bits de datos, sin paridad y 1 bit de parada.

Elemento	Valor en el proyecto	Comentario
Periférico	USART3	Ubicado en el bus APB1. Es una elección común porque deja libres otros UART para expansiones futuras.
TX	PB10 - AF7	Línea por la que el STM32 transmite bytes hacia la PC, un convertidor USB-UART o cualquier otro dispositivo serial.
RX	PB11 - AF7	Línea por la que el STM32 recibe bytes. El driver configura este pin con pull-up interno.
Formato	115200, 8N1	Velocidad suficientemente alta para depuración y telemetría sencilla, con configuración estándar fácil de usar desde una terminal serial.

3.1 Por qué PB10 y PB11

La tabla general del proyecto ya indica que la comunicación serial está asignada a PB10 y PB11. El driver respeta ese mapeo y configura ambos pines en función alterna AF7, que es la selección correspondiente a USART3 en el STM32F405. Esta elección no interfiere con el SPI del display, porque ese otro bloque usa PA5 y PA7.

4. Inicialización de USART3

La función `usart3_pb10_pb11_init_115200()` realiza toda la secuencia de arranque del puerto serial. El orden es correcto y didáctico: primero habilita relojes, luego configura pines y finalmente programa el periférico UART.

Fragmento 2. Primera mitad de la secuencia de inicialización

```
void usart3_pb10_pb11_init_115200(void)
{
    RCC->AHB1ENR |= RCC_AHB1ENR_GPIOBEN;
    RCC->APB1ENR |= RCC_APB1ENR_USART3EN;

    GPIOB->MODER &= ~( (3u << (10*2)) | (3u << (11*2)) );
    GPIOB->MODER |=  ( (2u << (10*2)) | (2u << (11*2)) );
    GPIOB->OTYPER &= ~( (1u << 10) | (1u << 11) );
    GPIOB->OSPEEDR |=  ( (3u << (10*2)) | (3u << (11*2)) );
    GPIOB->PUPDR &= ~( (3u << (10*2)) | (3u << (11*2)) );
    GPIOB->PUPDR |=  (1u << (11*2));
    GPIOB->AFR[1] &= ~( (0xFu << ((10-8)*4)) | (0xFu << ((11-8)*4)) );
    GPIOB->AFR[1] |=  ( (7u << ((10-8)*4)) | (7u << ((11-8)*4)) );
    ...
}
```

4.1 Configuración eléctrica de los pines

Ambos pines se ponen en modo Alternate Function. El tipo de salida queda en push-pull y la velocidad se lleva a un valor alto, lo cual es una práctica razonable para señales digitales rápidas. El detalle más interesante es el de PB11: como es entrada de recepción, el código activa un pull-up interno. Esto ayuda a que la línea permanezca en nivel alto cuando nadie está transmitiendo, lo cual coincide con el estado de reposo de la UART.

4.2 Programación del formato 8N1

Después de preparar los pines, el driver limpia CR1, CR2 y CR3. Al dejar estos registros en cero, la UART queda en su formato básico: 8 bits, 1 stop bit, sin paridad, sin control por hardware y sin DMA. Esta simplicidad es buena para clase porque evita capas de complejidad que todavía no son necesarias.

Fragmento 3. Configuración básica de registros en USART3

```
USART3->CR1 = 0;
USART3->CR2 = 0;
USART3->CR3 = 0;

const uint32_t PCLK1_HZ = 42000000u;
USART3->BRR = usart_brr_from_pclk(PCLK1_HZ, UART_BAUD);

USART3->CR1 |= USART_CR1_TE | USART_CR1_RE;
USART3->CR1 |= USART_CR1_UE;
```

4.3 Cálculo del baudio con oversampling por 16

La rutina auxiliar `usart_brr_from_pclk()` implementa el caso de sobremuestreo por 16, es decir, con `OVER8 = 0`. Bajo esa condición, el registro BRR codifica directamente `USARTDIV x 16`. El código usa una fórmula de redondeo entero para aproximar el divisor y obtener una velocidad lo más cercana posible a 115200 baudios.

Paso	Expresión	Resultado
Reloj del periférico	PCLK1 asumido en el driver	42 MHz
Baudios objetivo	UART_BAUD	115200
Divisor ideal	$PCLK1 / \text{baud}$	364.58 ticks por bit
Valor para BRR	$\text{round}(PCLK1 / \text{baud})$	365 decimal = 0x16D
Mantisa / fracción	22 / 13 en formato OVER16	$22 + 13/16 = 22.8125$

El punto didáctico clave aquí es que el driver no lee el árbol de reloj en tiempo real: simplemente asume que `PCLK1 = 42 MHz`. Eso es compatible con la configuración global actual del proyecto, donde el reloj del sistema es de 168 MHz y APB1 trabaja con prescaler 4. Si esa configuración cambiara en otra práctica, habría que actualizar también esta constante para no introducir error de baudio.

5. Rutinas de transmisión

El módulo ofrece tres primitivas relacionadas con transmisión: enviar un solo byte, enviar una cadena de caracteres y esperar hasta que la transmisión física termine por completo. Las tres están implementadas por polling, sin interrupciones ni colas intermedias.

Fragmento 4. Transmisión byte a byte y cadenas

```
void usart3_write_byte(uint8_t b)
{
    USART3->DR = b;
    while (!(USART3->SR & USART_SR_TXE)) {}
}

void usart3_write_str(const char *s)
{
    while (*s)
    {
        usart3_write_byte((uint8_t)*s++);
    }
}
```

5.1 Qué significa esperar por TXE

El bit TXE indica que el registro de datos está vacío y puede cargarse el siguiente byte. En esta implementación se escribe primero en DR y luego se espera a que TXE vuelva a activarse. Esa secuencia deja lista la UART para la siguiente escritura, por lo que funciona bien dentro de un envío repetitivo como `usart3_write_str()`.

5.2 Para qué sirve `uart3_flush_tc()`

El bit TC significa Transmission Complete. A diferencia de TXE, que solo habla del registro de datos, TC confirma que también el desplazamiento del último bit hacia la línea física ya terminó. Por eso `uart3_flush_tc()` es útil cuando se quiere asegurar que el último byte realmente salió del pin antes de apagar el periférico, cambiar de modo de operación o liberar un transceiver externo.

Fragmento 5. Espera del final físico de transmisión

```
void uart3_flush_tc(void)
{
    while(!(USART3->SR & USART_SR_TC)) {}
}
```

6. Rutinas de recepción

La recepción se presenta en dos variantes: una no bloqueante y otra bloqueante. Esta dualidad es muy útil en clase porque enseña dos estilos de programación distintos sobre el mismo periférico.

Fragmento 6. Recepción no bloqueante y bloqueante

```
int usart3_read_byte_nonblocking(uint8_t *out)
{
    if (USART3->SR & USART_SR_RXNE)
    {
        *out = (uint8_t)(USART3->DR & 0xFFu);
        return 1;
    }
    return 0;
}

uint8_t usart3_read_byte_blocking(void)
{
    while (!(USART3->SR & USART_SR_RXNE)) {}
    return (uint8_t)(USART3->DR & 0xFFu);
}
```

6.1 Recepción no bloqueante

La función `usart3_read_byte_nonblocking()` revisa el bit RXNE. Si hay dato disponible, lo lee desde DR, lo almacena en la variable apuntada por `out` y devuelve 1. Si no hay dato, simplemente devuelve 0. Esta forma es ideal para lazos principales donde no se desea detener el resto del sistema mientras se espera tráfico serial.

6.2 Recepción bloqueante

La variante `usart3_read_byte_blocking()` se detiene hasta que llegue un byte. Su código es mínimo y muy claro, por lo que funciona bien como primer ejemplo didáctico. La desventaja es que, si se llama dentro de

un lazo crítico, el procesador puede quedar detenido indefinidamente hasta que un emisor externo envíe información.

6.3 Detalle importante: lectura del DR

En ambos casos, leer USART3 ->DR no solo entrega el dato recibido; también sirve para limpiar la condición de recepción pendiente. Este comportamiento de los periféricos seriales es importante para los estudiantes porque muestra que en hardware embebido la lectura de un registro puede tener un efecto funcional, no solo informativo.

7. Capa de interfaz uart_if

La capa de interfaz repite el patrón usado en otros módulos del proyecto. En lugar de invocar directamente funciones sueltas, la aplicación puede trabajar con una estructura constante cuyas entradas apuntan a las operaciones del driver.

Fragmento 7. Estructura pública de uart_if.h

```
typedef struct {
    void (*init_115200)(void);
    void (*write_byte)(uint8_t b);
    void (*write_str)(const char *s);
    int (*read_byte_nonblocking)(uint8_t *out);
    uint8_t (*read_byte_blocking)(void);
    void (*flush_tc)(void);
} uart_if_t;
```

Fragmento 8. Instancia concreta creada en uart_if.c

```
const uart_if_t uart_STM32 = {
    .init_115200 = uart3_pb10_pb11_init_115200,
    .write_byte = uart3_write_byte,
    .write_str = uart3_write_str,
    .read_byte_nonblocking = uart3_read_byte_nonblocking,
    .read_byte_blocking = uart3_read_byte_blocking,
    .flush_tc = uart3_flush_tc
};
```

Esta decisión no es obligatoria en un firmware pequeño, pero sí es muy valiosa para enseñanza. Obliga a pensar el módulo como una abstracción con servicios bien definidos y prepara a los estudiantes para arquitecturas más grandes donde distintos drivers pueden compartir una misma interfaz lógica.

8. Uso real en la aplicación

En el estado actual del proyecto, main.c solo llama a la inicialización del puerto serial. Eso significa que el canal queda listo desde el arranque, aunque todavía no se aprovechan las funciones de transmisión o recepción dentro del lazo principal.

Fragmento 9. Punto donde se inicializa USART3 dentro del arranque

```
int main(void)
{
    clear_fault_flags();
    FPU_Enable();

    SYSClk_STM32.init();
    ini_pantalla();
    quad_STM32.init();
    PWM_STM32.motor.init(PWM_TARGET_HZ);
    uart_STM32.init_115200();
    GPIO_STM32.motor.init();
    GPIO_STM32.rgb.init();
    ...
}
```

Aunque aquí la UART aún no se usa para imprimir variables, el módulo sería ideal para ampliar el proyecto con mensajes de diagnóstico como rpm, ye, error del estimador o parámetros del modelo. También permitiría capturar comandos simples desde una terminal, por ejemplo para variar el duty del motor o reiniciar la matriz de covarianza del RLS sin recompilar.

9. Ideas didácticas y observaciones importantes

Punto	Explicación para clase
Polling en vez de interrupciones	Todo el módulo opera mediante sondeo de banderas del registro SR. Es simple de entender, pero no escala igual de bien que una implementación con interrupciones o DMA cuando el tráfico aumenta.
Constante PCLK1_HZ	El baudio depende de la constante local de 42 MHz. Si la configuración del reloj del sistema cambia, la UART dejará de estar exactamente en 115200 a menos que se actualice esa constante.
Sin manejo explícito de errores	El código no revisa banderas como ORE, FE o NE. Para una primera práctica está bien, pero más adelante convendría discutir recepción robusta.
TXE vs TC	Diferenciar ambos bits es una excelente oportunidad didáctica: uno habla del registro de datos y el otro del final real de la transmisión en la línea.
Interfaz reusable	La estructura <code>uart_if_t</code> deja la puerta abierta para construir otras implementaciones seriales compatibles sin modificar el código de alto nivel.
Módulo listo para crecer	Aunque hoy la UART solo se inicializa, ya existe una base completa para telemetría, consola de depuración o adquisición serial de comandos.

10. Conclusión

El módulo de UART sirve muy bien como material de enseñanza porque reúne configuración de pines, programación de registros seriales, manejo de banderas y abstracción por interfaz. Aunque en esta versión del firmware solo se usa en la fase de inicialización, el código ya deja lista una base útil para depuración, telemetría y experimentación. También permite explicar conceptos esenciales como baudios, formato 8N1, sondeo de banderas, diferencia entre TXE y TC, y relación entre el árbol de reloj y el registro BRR.